

# Laughing Waters Harvesting - User & Admin Manual

---

Laughing Waters Harvesting is a small, self-hosted app that tracks litchis from the moment they're picked in the field, through dispatch to the pack house, to receiving, wages, and reporting. It runs on one local server and is used from several devices at once - a phone or tablet at each field station, one or two devices at the pack house gate, and a computer in the farm office.

This manual is for everyone who touches the app: - **Field capture staff** - picking teams who log crates as they're picked. - **Pack house staff** - whoever receives incoming loads at the gate. - **Farm admin / office staff** - whoever manages workers, wages, reports, and the server itself.

# Table of Contents

---

1. [Overview & Concepts](#)
  2. [Initial Server Setup](#)
  3. [Device Setup](#)
  4. [Field App - Capturing the Harvest](#)
  5. [Worker ID Badges](#)
  6. [Pack House Receiving](#)
  7. [Admin - Dashboard](#)
  8. [Admin - Master Data](#)
  9. [Admin - Payments](#)
  10. [Admin - Reports](#)
  11. [Admin - Settings](#)
- [Troubleshooting / FAQ](#)
- [Annexe A: Data Field Reference](#)
-

# 1. Overview & Concepts

---

## The three device roles

Every device that opens the app is one of exactly three roles:

| Role       | Used by                                     | What it does                                                                                |
|------------|---------------------------------------------|---------------------------------------------------------------------------------------------|
| Field      | Picking teams, one device per field station | Logs crates as they're picked, sends "picking slips" (loads) to the pack house              |
| Pack House | Gate/receiving staff                        | Sees incoming loads, checks them in when the truck arrives                                  |
| Admin      | Farm office                                 | Manages workers/teams/blocks/suppliers, calculates wages, runs reports, configures settings |

A device only ever shows the screen for its own role - a field tablet never sees the admin screens, and vice versa. This keeps each device simple and hard to misuse by accident.

## Why capture is offline-first

Field stations are often out of signal range. The field app is built so that **capturing a crate never depends on having a connection** - every crate is saved on the device first, and quietly synced to the server in the background whenever a connection is available. A picking team can keep working through a signal dropout without losing anything or needing to notice it happened.

## Why the oldest-first, color-coded ordering matters

Litchis are highly perishable - quality drops fast once picked, and drops faster again once picked fruit sits waiting in the sun. Two places in the app use the same "traffic light" idea to make that visible at a glance:

- The **pack house's incoming queue** shows the oldest (longest-waiting) load first, colored **green** → **yellow** → **red** as it ages past the thresholds set in Settings (see [chapter 11](#)).
- The **admin Dashboard's Harvesting and In Transit lists** use the exact same coloring, so the office can see at a glance whether fruit is moving through fast enough.

This isn't just a UI nicety - it's the app's main quality-control signal: red means "this has been waiting too long, offload/process it before anything green."

---

## 2. Initial Server Setup

---

The app is a single Python service (FastAPI) that serves its own frontend and stores everything in a local SQLite database file - there's no separate database server, cloud account, or build step involved. It runs on one ordinary computer in the farm office (the "server"), and every other device (field phones/tablets, pack house devices) just opens a web address pointing at that computer - nothing needs to be installed on those other devices.

### Prerequisites

- A Windows, Mac, or Linux computer that can be left switched on and connected to the network for the whole harvest season - this is the "server" referred to throughout this manual. It does not need to be powerful; any normal office PC is enough.
- Python 3.9 or newer (3.11 is a safe default if installing fresh).
- Network access from every device that needs to reach the app (the same Wi-Fi/LAN the farm already uses, or a Tailscale network if devices need to reach it from outside the farm's own network - the app itself doesn't care which, it just needs to be reachable).
- The app's project folder, copied onto that computer (via USB drive, email/zip, or `git clone` - however it was given to you). This manual assumes it ends up at `C:\LaughingWaters` on Windows, or `~/LaughingWaters` on Mac/Linux; adjust the paths below if you use a different location.

### Setting up on a Windows PC (step by step)

This is the most common setup, since most farm offices run Windows. Every step is done once, when first setting up the server.

**Step 1 - Copy the app folder onto the PC.** Place the whole project folder somewhere permanent and easy to find, e.g. `C:\LaughingWaters`. Avoid Desktop or Downloads, since those are more likely to get tidied up or deleted by accident.

#### Step 2 - Install Python.

1. Go to `python.org` in a browser and download the latest Python 3 installer for Windows (64-bit).
2. Run the installer. **On the very first screen, tick the checkbox at the bottom that says "Add python.exe to PATH"** before clicking "Install Now" - this step is easy to miss and, if skipped, every command below will fail with `'python' is not recognized`.
3. Once installation finishes, click "Close".

#### Step 3 - Open Command Prompt in the backend folder.

1. Open the `C:\LaughingWaters\backend` folder in File Explorer.
2. Click into the empty area of the address bar at the top of the window, type `cmd`, and press Enter. A black Command Prompt window opens already pointed at that folder (confirm the prompt reads `C:\LaughingWaters\backend>`).

**Step 4 - Create a virtual environment.** A virtual environment keeps this app's Python packages separate from anything else on the PC. In the Command Prompt window, run:

```
python -m venv .venv
```

This creates a `.venv` folder inside `backend` and takes a few seconds.

**Step 5 - Activate the virtual environment.**

```
.venv\Scripts\activate
```

The prompt changes to start with `(.venv)` once this has worked - check for that before continuing. You'll need to repeat this activation step every time you open a new Command Prompt window to work with the app (but *not* every time the server itself runs day-to-day - see Step 10).

**Step 6 - Install the app's dependencies.**

```
pip install -r requirements.txt
```

This downloads everything the app needs (FastAPI, the database library, etc.) into the virtual environment. It can take a few minutes on the first run, depending on the internet connection. If it fails partway with a build/compiler error, the most common cause is a 32-bit Python install - uninstall it and reinstall the 64-bit version from Step 2.

**Step 7 - Run the server for the first time.**

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

- `--host 0.0.0.0` makes the server reachable from other devices on the network, not just this PC - this is required for field/pack house devices to connect.
- `8000` is just an example port; any free port works, but stick with one number once devices are configured against it.
- The window will print a few lines ending in something like `Uvicorn running on http://0.0.0.0:8000` and then go quiet - that's normal; it means the server is up and waiting.  
**Leave this Command Prompt window open** - closing it stops the server.

**Step 8 - Confirm it's working.** On the same PC, open a browser and go to `http://localhost:8000/`. You should see the app's device setup screen (see [chapter 3](#)). If you see this, the server itself is working correctly.

**Step 9 - Find this PC's network address.** Other devices don't use `localhost` - they need this PC's actual address on the network. Open a **second** Command Prompt window (leave the server running in the first one) and run:

```
ipconfig
```

Look for **"IPv4 Address"** under the network adapter that's actually connected (Wi-Fi or Ethernet) - it looks like `192.168.1.50`. Other devices will reach the app at `http://192.168.1.50:8000/` (using this PC's own address and the port from Step 7).

**Step 10 - Allow the app through Windows Firewall.** Windows will usually pop up a "Windows Defender Firewall has blocked some features of this app" prompt the first time the server starts - tick both "Private networks" and "Public networks" (if shown) and click "Allow access". If that prompt was missed or dismissed, add the rule manually:

1. Open **Windows Security** → **Firewall & network protection** → **Advanced settings**.
2. Click **Inbound Rules** → **New Rule...**
3. Choose **Port** → Next.
4. Choose **TCP**, and under "Specific local ports" enter the port from Step 7 (e.g. `8000`) → Next.
5. Choose **Allow the connection** → Next.
6. Leave Domain/Private/Public all ticked → Next.
7. Give it a name, e.g. "Laughing Waters Server" → Finish.

**Step 11 - Stop the PC from going to sleep.** The server only works while the PC is awake. Go to **Settings** → **System** → **Power & battery** → **Screen and sleep**, and set "When plugged in, put my device to sleep" to **Never**. (The screen itself can still turn off - that doesn't affect the server - only "sleep"/"hibernate" does.)

**Step 12 - (Recommended) Make the server start automatically.** Without this step, someone has to manually repeat Steps 3, 5, and 7 every time the PC restarts (e.g. after a power cut or Windows update). To avoid that, first create a new text file at `C:\LaughingWaters\start_server.bat` containing exactly:

```
@echo off
cd /d "C:\LaughingWaters\backend"
call .venv\Scripts\activate.bat
uvicorn main:app --host 0.0.0.0 --port 8000
```

Then set Windows to run it automatically on startup:

1. Open **Task Scheduler** (search for it in the Start menu).
2. Click **Create Basic Task...**, name it "Laughing Waters Server", Next.
3. Trigger: choose **"When the computer starts"**, Next.
4. Action: choose **"Start a program"**, Next, then browse to and select `C:\LaughingWaters\start_server.bat`, Next, Finish.
5. Find the new task in the Task Scheduler Library, right-click → **Properties**, and on the **General** tab tick **"Run whether user is logged on or not"** so it starts even before anyone signs in. You'll be asked for the Windows account password when you save this.

- Restart the PC once to confirm the server comes up on its own (check from another device by browsing to `http://<this-pc's-IP>:8000/`).

With this in place the server behaves like any other piece of office equipment - it just needs the PC left on.

## Setting up on Mac or Linux

From the backend/ folder:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Then, to run it:

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

- `--host 0.0.0.0` makes the server reachable from other devices on the network, not just the machine it's running on.
- Find this machine's network address with `ifconfig` (look for `inet` under the active Wi-Fi/Ethernet adapter) so other devices can reach `http://<that-address>:8000/`.
- To keep the server running after closing the terminal, and to have it restart automatically after a reboot, use your platform's standard approach for long-running services (e.g. a `launchd` agent on Mac, or a `systemd` service on Linux) pointed at the same `uvicorn` command above.

## Connecting external users with Tailscale (only if needed)

Skip this section entirely if every device (field, pack house, admin) stays on the farm's own Wi-Fi/LAN - it's only needed if someone **outside** that network has to reach the server, e.g. a remote bookkeeper checking Reports, an off-site owner checking the Dashboard, or a partner farm's own admin accessing their supplier data from elsewhere. Tailscale is a free, private network that lets specific outside devices reach the server securely over the internet, without opening any ports on the farm's router or exposing the app to the public internet.

### Step 1 - Install Tailscale on the server.

- On the same PC the server runs on, go to `tailscale.com/download` in a browser and download the installer for its operating system (Windows, Mac, or Linux).
- Run the installer, then sign in when prompted (with a Google, Microsoft, or email account) - this creates your farm's own private Tailscale network (a "tailnet"), separate from anyone else's.
- Once signed in, Tailscale assigns this server a private address like `100.x.x.x`. Find it by clicking the Tailscale icon in the system tray (Windows) or menu bar (Mac) and reading the address shown there, or by running `tailscale ip -4` in a Command Prompt/terminal.

4. Make sure Tailscale is set to start automatically: right-click its tray icon → **Preferences/Settings** → confirm "**Start on login**" (or equivalent) is ticked, so it reconnects on its own after every restart, the same way the server itself does ([Step 12](#) on Windows).

## Step 2 - Share (not invite) access to just this one machine.

Tailscale's "**Share**" feature grants an outside person access to only this one server, without adding them to the rest of the farm's tailnet or letting them see any other device on it - this is the right option for an external user, as opposed to "Invite" (which adds someone as a full member of the tailnet).

1. In a browser, go to [login.tailscale.com/admin/machines](https://login.tailscale.com/admin/machines) and sign in with the same account used in Step 1.
2. Find this server in the machine list, click the "..." menu next to it, and choose **Share...**
3. Enter the external person's email address and send the invite (or copy the generated link and send it yourself, e.g. via WhatsApp or email).
4. The external person accepts the invite, creating their own free Tailscale account if they don't already have one, and installs Tailscale on their own phone, tablet, or PC (same download step as Step 1, but signing in with their own account).

**Step 3 - Connecting.** Once accepted, the external device can reach this one server at its Tailscale address from anywhere with an internet connection - e.g. <http://100.x.x.x:8000/> - exactly the same way a device on the farm's own Wi-Fi reaches it by LAN address. Everything else (the device setup screen, roles, admin login) works identically; Tailscale only changes how the device reaches the server, not what it can do once it's there.

**Step 4 - Removing access later.** When an external person no longer needs access (e.g. a contractor's season is over), go back to [login.tailscale.com/admin/machines](https://login.tailscale.com/admin/machines), find their device under the server's sharing settings, and remove it - this revokes their access immediately without affecting anyone else.

## Keeping the server running

However it's started, leave it running continuously during harvest season - the app expects to be a long-lived local service, not something started and stopped around each use. (The automatic nightly backup, [chapter 11](#), only fires if the server happens to be running at 02:00 - see that chapter's note on this limitation.)

## What happens on first startup

The very first time the server runs, it automatically creates the database and seeds a clean starting baseline:

- Two teams: **Span A** and **Span B** (indunas left blank - fill in via [Master Data](#))
- The farm's **18 real block labels** (7, 8a, 8b, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 22, 23, 34, 35) - variety, tree count, and hectares are left blank for the admin to fill in, since they weren't safe to guess automatically



- **8 devices:** device-01 through device-05 (field), device-06 and device-07 (pack house), and admin-pc (admin) - see [chapter 3](#) for how these get assigned to physical devices
- A default wage rate of **R3.00/kg**
- One supplier row representing the farm's own fruit ("Laughing Waters (Own)")
- A default admin login: **username admin, password ChangeMe123!**

■■ **Change the default admin password immediately** after first login, via [Settings](#) → [Change admin password](#).

## ■■ **Do not run `seed_demo.py` on a real farm database**

The repo includes `backend/seed_demo.py`, which fills the database with **fake historical harvest data** for demoing and testing the app. It is **not** part of normal startup and must never be run against a real farm's database - doing so will inject invented workers, lots, and payments alongside real data with no easy way to tell them apart afterward. It's safe to use only against a throwaway/test copy of the app.

---

### 3. Device Setup

---

The very first time any device (phone, tablet, or computer) opens the app's URL, it shows a one-time **Device Setup** screen instead of jumping straight into a role's screen.

1. Enter the **Device ID** you were given by the admin (a dropdown of the preseeded IDs - device-01 through device-07, admin-pc - is offered, but the admin may have added more via [Master Data → Devices](#)).
2. Tap **Continue**.
3. The device looks up that ID's role and **automatically routes itself** to the right screen - Field, Pack House, or Admin - based on what's configured for that device.
4. The device **remembers its ID** after this (stored in the browser, not on the server) - it won't ask again on future visits, and will jump straight to its role's screen.

#### "Unknown device id"

If the entered ID isn't recognized, the device shows an error and refuses to continue. **Devices are never auto-registered** - an admin must create the device first, in [Master Data → Devices](#), before it can be used. This is intentional: it stops a stray or mistyped device ID from silently attaching itself to the wrong team or role.

#### Reassigning a device

To point a device at a different role/station (e.g. repurposing a spare tablet), either: - Clear the device's browser data/cache so it forgets its saved ID and shows the setup screen again, or - Simply change that device ID's role/station in [Master Data → Devices](#) - the physical device keeps using the same ID, but the server now treats it differently on its next check-in.

---

## 4. Field App - Capturing the Harvest

---

The field app is what a picking team's device shows once set up. Its job is simple: log every crate as it's weighed, and send a "picking slip" (a load of crates) to the pack house when a truck is ready to take it.

### Station header

The top of the screen shows which station/team/induna this device is registered as, and a sync status indicator: **Offline**, **Online - synced**, **Syncing...**, or **Online - sync failed, retrying**.

### Identifying the worker: QR scan only

Tap **Scan Worker QR** and point the device's camera at the worker's printed ID badge (see [chapter 5](#) for how badges are made). The app matches the scanned code against known workers and shows the worker's name once matched.

**Why QR-only, no typing/dropdown?** Picking teams can move dozens of workers through a station in a day - manually finding the right name in a long dropdown list, especially with common surnames, risks misattributing a crate (and therefore wages) to the wrong person. A QR scan is unambiguous.

If the scanned code doesn't match any known worker, the app tells you so and nothing is selected - print or reprint that worker's badge instead of guessing.

### Selecting the block

Choose the block being picked from the **Block** dropdown.

### Capturing a crate's weight

Use the on-screen numeric keypad to enter the crate's weight in kg, then tap **Save Crate**. The crate is saved immediately to the device - see "Offline-first capture" below - and the running totals update:

- **Crate count** and **total weight** for the current, not-yet-dispatched load
- **Elapsed time**, color-coded green/yellow/red the same way described in [chapter 1](#), so the team can see for themselves when a load has been sitting long enough that it should go.

### Offline-first capture

Every crate is written to the device's local storage the instant you tap Save Crate - **capture never waits on a network connection**. In the background, the app checks for a connection every few seconds and quietly syncs anything not yet sent to the server. You do not need to do anything to make this happen, and a signal dropout mid-picking has no effect on capture.

### Sending the load: "Send Picking Slip"

When a truck is ready to take the crates picked so far, tap **Send Picking Slip**. You'll be asked for:

- **Crates going now** - defaults to every crate captured so far, but can be reduced if the truck can't take everything (see "Splitting a load" below)
- **Driver name** - required

Once sent, the load moves into "in transit" and will appear in the pack house's queue (see [chapter 6](#)). The screen shows a confirmation banner ("Picking Slip Sent - X crates - Y kg dispatched"), and the load also appears under **Dispatched Lots Today** on the same screen.

## Splitting a load

Sometimes a truck arrives before a picking round is finished. If you enter a **Crates going now** value lower than the total captured, the app splits the load: the crates going now are dispatched immediately under the current slip number, and the remaining crates are automatically rolled onto a **new** slip number, ready to combine with whatever gets picked next.

Splitting requires an active connection (the server needs to be the one deciding which crates go on which slip). If offline, the app shows a hint and asks you to either reconnect or just send everything on this load instead of splitting.

The pack house will see both resulting loads flagged as a **split load** with a link back to the other part, so receiving staff know part of this same picking session may arrive separately (see [chapter 6](#)).

---

## 5. Worker ID Badges

---

Badges are what a field device scans to identify a worker (see [chapter 4](#)). An admin generates and prints them from **Master Data** → **Workers**:

- **Print Badges (filtered)** - only the workers currently matching the Farm/Supplier filter
- **Print Badges (selected)** - only the workers with their row checkbox ticked
- **Print Badges (all)** - every active worker

Each badge shows the farm name, the worker's photo (if one's been captured - see [chapter 8](#)), a QR code encoding their employee number, their name, and their employee number printed in large text. Badges print at roughly 9cm × 7cm, several to a page, ready to laminate.

---

## 6. Pack House Receiving

---

This is the screen at the gate where incoming loads are checked in.

### The in-transit queue

Every load currently on its way (dispatched from a field station, or logged as an external delivery - see below) appears here, **oldest first**, colored green/yellow/red by how long it's been in transit (the same thresholds as [chapter 1](#), configurable in [Settings](#)). Each card shows the slip number, farm/ supplier, team, driver, crate count, and total kg.

If a load is part of a **split** (see [chapter 4](#)), its card shows a "**Split load - N related slip(s)**" flag. Opening the load shows the related slip(s) with their own status (still in transit, already received, or still being picked) - so receiving staff know to expect (or ask about) the rest of that picking session.

### Logging an external delivery

For fruit arriving from another farmer (not via this farm's own field devices), tap **+ Log External Delivery** and fill in:

- **Supplier** - the external farm this fruit belongs to (must already exist in [Master Data → Suppliers](#))
- **Crates**
- **Total Kg**
- **Driver**
- **Notes**

This drops the load straight into the in-transit queue like any other load, ready to be received the same way.

### Receiving a load

Tapping a card opens the **Receive** modal, with fields in this order:

1. **Expected crates** - read-only, what was dispatched
2. **Actual crates received** - enter what actually arrived
3. **Condition** - tick any that apply: **Good**, **Damaged**, **Sunburn**, **Wet**, **Other** (more than one can be ticked; all ticked values are recorded together)
4. **Notes**
5. **Received by** - required; the app remembers the last name entered here and prefills it next time, to save retyping for the same gate staff member

Confirming moves the load's status to **received** and records the exact receiving time - this is what feeds the admin Dashboard/Reports' "Received" figures.

---

## 7. Admin - Dashboard

---

The Dashboard is the admin app's landing screen - a farm-wide overview of what's happening right now, all scoped to a shared filter bar at the top.

### Filter bar

- **Farm / Supplier** - narrow everything below to one farm/supplier, or leave on "All farms / suppliers"
- **Period start / Period end**, plus quick-fill buttons **Today**, **This Week**, **Season** (season = 1 Jan - 31 Dec of the harvest year set in [Settings](#)) - or set custom dates directly
- **Refresh** - re-fetches everything below using the current filter values

### KPI cards

Teams Active, Workers Active, Blocks Active (all counted as "had activity in the selected period", not a static list), Total Kg, Total Crates, Avg Kg/Lot, Avg Kg/Crate, and a breakdown of Harvesting / In Transit / Received crates and kg.

### The five collapsible lists

Each starts collapsed, showing its running total in the header; tap to expand for the detail rows.

1. **Harvesting** - loads still being picked (not yet dispatched), oldest-first, color-coded the same as pack house
  2. **In Transit** - dispatched, not yet received, same ordering/coloring
  3. **Received** - newest-received first
  4. **Workers** - per-worker crates/kg/amount-due/avg-kg-per-crate, sorted by kg picked (highest first), showing each worker's **Farm/Supplier**
  5. **Blocks** - per-block crates/kg/avg-kg-per-crate/avg-kg-per-tree
-

## 8. Admin - Master Data

---

Master Data has five subtabs for the farm's reference data.

### Workers

Employee number, name, ID number, bank/account, WhatsApp number, which farm/supplier they belong to, a photo (captured via the device's camera right in the edit form, or uploaded), and active/inactive. Supports CSV/xlsx **Export** and **Import** for bulk edits, and the [Print Badges](#) buttons.

### Teams

ID (e.g. "A"), name (e.g. "Span A"), induna, active.

### Blocks

The 18 preseeded block IDs (see [chapter 2](#)) plus name, variety, tree count, hectares, active. Supports CSV/xlsx export/ import, useful for filling in variety/trees/hectares in bulk rather than one block at a time.

### Devices

ID, role (field/packhouse/admin), station name, team, induna, data capturer, active. This is where new devices must be added before they can complete [Device Setup](#).

### Suppliers

Every farm whose fruit passes through this pack house, including the farm's own row (marked "(Own Farm)"). Contact details and a packing rate (per kg, or per crate if per-kg is left at 0) used for the **Facility Billing** panel on this same subtab - pick a supplier and date range, tap **Calculate**, and see how much facility-use fee that supplier owes for fruit actually received (not still in transit) in that period.

---



## 9. Admin - Payments

---

Calculates and exports wages for a filtered period.

- Same filter bar convention as the Dashboard (Farm/Supplier + Today/This Week/Season/custom dates)
- **Calculate Wages** builds the table below, **grouped by farm/supplier** (own farm first, then alphabetically), each group showing a summary row (worker count, total kg, total wages) followed by that group's workers (name, kg, rate, amount due)
- **Export Wage Sheet** downloads the same grouped breakdown as an `.xlsx` file

This screen only calculates what's **owed** - marking wages as paid, or tracking payment status, is deliberately not handled inside this app; that's managed in whatever external system/process the farm already uses for actually paying workers.

---

## 10. Admin - Reports

---

Downloadable .xlsx reports, all sharing the same filter bar as Payments/ Dashboard (Farm/Supplier + date range):

| Report                 | Contents                                                               |
|------------------------|------------------------------------------------------------------------|
| Daily Harvest Summary  | Crates/kg by block and team for one day                                |
| Lot & Receiving Report | Every lot dispatched in the range, with receiving detail once received |
| Harvesting List        | Loads still being picked, matching the Dashboard's Harvesting list     |
| In Transit List        | Dispatched, not-yet-received loads                                     |
| Received List          | Received loads                                                         |
| Worker Harvest Report  | Per-worker crates/kg/amount-due/avg-kg-per-crate                       |
| Block Harvest Report   | Per-block crates/kg/avg-kg-per-crate/avg-kg-per-tree                   |

---

## 11. Admin - Settings

---

### Data Backup

- **Backup Now** - immediately zips the database and worker photos into a downloadable archive, and adds it to the list below (created date, size, a download link per entry)
- An identical backup also runs **automatically every day at 02:00**, keeping only the **14 most recent** backups (older ones are deleted automatically)

■■ The automatic backup only runs if the server happens to be running at 02:00. If the machine is switched off overnight, that night's backup simply doesn't happen - there's no separate always-on scheduler behind it. Keep the server running continuously during harvest season (see [chapter 2](#)).

**Recommended: copy backups off the server regularly.** The 14-backup retention only protects against recent mistakes (e.g. accidentally deleting a worker) - it does **not** protect against the server's disk failing entirely, since all 14 copies live on that same machine. Every so often, download the latest backup from this list and copy it somewhere off the machine - a cloud drive such as Google Drive, Dropbox, or similar is a simple, effective option. That off-machine copy is the only real safeguard against losing everything if the hardware fails.

### Farm settings

Farm name, location description, current harvest season (year - drives what "Season" means throughout the app), the green→yellow and yellow→red urgency thresholds in minutes (used everywhere the traffic-light coloring appears - [chapter 1](#)), and GPS coordinates (latitude/longitude, or pick a location on the map) - setting these enables automatic weather capture on every dispatched load.

### Harvest rate

The per-kg wage rate used by [Payments](#).

### Change admin password

Change the admin login password - **do this immediately after first setup** ([chapter 2](#)).

### Header weather

The admin screen's header shows a live current-weather readout (temperature, condition, humidity) next to the farm name/clock - a quick at-a-glance check of conditions without leaving the app.

---

## 12. Troubleshooting / FAQ

---

**"Unknown device id" on a device's first setup** The device ID hasn't been created yet. An admin must add it in [Master Data → Devices](#) first - see [chapter 3](#).

**"QR code doesn't match a known worker" when scanning a badge** Either the worker doesn't exist in [Master Data → Workers](#) yet, or their badge is stale/misprinted. Reprint the badge from Workers after confirming the worker record exists (see [chapter 5](#)).

**Field app shows "Online - sync failed, retrying"** The device has a connection but the server rejected or couldn't be reached for the last sync attempt. Nothing is lost - captured crates stay queued on the device and the app keeps retrying automatically every few seconds.

**"Reconnect to send a partial load, or dispatch everything now" when sending a picking slip** You tried to split a load (send fewer crates than captured) while offline. Splitting needs a connection because the server decides the split; either wait for a connection or send the full load instead.

**Forgotten admin password** There's currently no self-service "forgot password" flow in the app - a new password can only be set from inside Settings while already logged in. If the admin password is lost entirely, restoring access requires direct access to the server's database rather than anything available from the app's screens.

---

## Annexe A: Data Field Reference

Field-by-field reference for every stored record type, with a realistic example value and any limitation worth knowing. Types/defaults/foreign keys below match `backend/models.py` exactly.

### Worker

| Field                        | Type              | Example         | Notes / Limitations                                                                                        |
|------------------------------|-------------------|-----------------|------------------------------------------------------------------------------------------------------------|
| <code>id</code>              | text              | "001"           | Primary key - the employee number. Must be unique and must match the number printed on the worker's badge. |
| <code>first_name</code>      | text              | "Sipho"         |                                                                                                            |
| <code>last_name</code>       | text              | "Dlamini"       |                                                                                                            |
| <code>id_number</code>       | text              | "8501015800083" | Free text, not validated.                                                                                  |
| <code>bank</code>            | text              | "Nedbank"       |                                                                                                            |
| <code>account</code>         | text              | "9963334018"    |                                                                                                            |
| <code>whatsapp_number</code> | text              | " +27821234567" | Optional, not currently used to send anything automatically.                                               |
| <code>supplier_id</code>     | number (optional) | 2               | Which farm this worker belongs to. Left blank for the farm's own workers.                                  |
| <code>photo_filename</code>  | text              | "001.jpg"       | Set automatically when a photo is captured/uploaded - not hand-edited.                                     |
| <code>active</code>          | true/false        | true            | Inactive workers are hidden from most pickers/dropdowns but kept for history.                              |

### Team

| Field               | Type | Example          | Notes / Limitations                            |
|---------------------|------|------------------|------------------------------------------------|
| <code>id</code>     | text | "A"              | Primary key. Short code, e.g. a single letter. |
| <code>name</code>   | text | "Span A"         |                                                |
| <code>induna</code> | text | "Samuel Mthembu" |                                                |

|        |            |      |  |
|--------|------------|------|--|
| active | true/false | true |  |
|--------|------------|------|--|

## Block

| Field    | Type       | Example     | Notes / Limitations                                                                                   |
|----------|------------|-------------|-------------------------------------------------------------------------------------------------------|
| id       | text       | "8a"        | Primary key - a real farm block label. One of the 18 preseeded labels (see chapter 2); not free-form. |
| name     | text       | "Block 8a"  |                                                                                                       |
| variety  | text       | "Mauritius" |                                                                                                       |
| trees    | number     | 450         | Whole number of trees on the block.                                                                   |
| hectares | decimal    | 2.3         |                                                                                                       |
| active   | true/false | true        |                                                                                                       |

## Device

| Field          | Type            | Example           | Notes / Limitations                                                                      |
|----------------|-----------------|-------------------|------------------------------------------------------------------------------------------|
| id             | text            | "device-01"       | Primary key - must be entered exactly on the physical device's setup screen (chapter 3). |
| role           | enum            | "field"           | Must be exactly one of field, packhouse, admin.                                          |
| station        | text            | "Field Station 1" |                                                                                          |
| team_id        | text (optional) | "A"               | Which team this field device belongs to. Not applicable to pack house/admin devices.     |
| induna         | text            | "Samuel Mthembu"  |                                                                                          |
| data_capture_r | text            | ""                | Free text, optional.                                                                     |
| active         | true/false      | true              |                                                                                          |
| last_seen      | timestamp       | (auto)            | Updated automatically every time the device checks in - not editable.                    |

## Supplier

| Field                                              | Type       | Example                               | Notes / Limitations                                                                                                                     |
|----------------------------------------------------|------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| id                                                 | number     | 1                                     | Primary key, auto-assigned.                                                                                                             |
| name                                               | text       | "Jansen Boerdery"                     |                                                                                                                                         |
| contact_name /<br>contact_phone /<br>contact_email | text       | "Piet Jansen" /<br>"082-555-1234" / — | All optional.                                                                                                                           |
| is_own_farm                                        | true/false | false                                 | Exactly one supplier row in the whole system should ever have this set to <code>true</code> - that row represents the farm's own fruit. |
| packing_rate_per_kg                                | decimal    | 1.50                                  | If greater than 0, this rate is used for Facility Billing; otherwise the per-crate rate below is used.                                  |
| packing_rate_per_crate                             | decimal    | 25.00                                 | Only used when the per-kg rate is 0.                                                                                                    |
| active                                             | true/false | true                                  |                                                                                                                                         |

## HarvestRecord (one crate)

| Field     | Type               | Example              | Notes / Limitations                                                                                                                             |
|-----------|--------------------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| uuid      | text               | (auto-generated)     | Primary key, generated on the capturing device - lets a retry after a dropped connection safely resubmit the same crate without duplicating it. |
| timestamp | timestamp          | 2026-07-08T09:14:00Z | When the crate was captured.                                                                                                                    |
| worker_id | text<br>(optional) | "001"                | Set via the QR badge scan (chapter 4) - required in practice even though nullable in the schema.                                                |
| block_id  | text<br>(optional) | "8a"                 | Required in practice.                                                                                                                           |
| weight_kg | decimal            | 12.4                 | A single crate's weight - typically in the 8-20kg range for litchi crates.                                                                      |

|              |                   |     |                                                                                                                                                                                     |
|--------------|-------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| deduction_kg | decimal           | 0.0 | Exists in the schema for a future "afrekkings" (waste/reject deduction) workflow, but there is currently no screen to enter a non-zero value - always 0 in this version of the app. |
| lot_id       | number (optional) | 21  | Always set at capture time, pointing at a placeholder load that becomes a real dispatch once "Send Picking Slip" is used.                                                           |

## Lot (a picking slip / load)

| Field                                               | Type                     | Example                    | Notes / Limitations                                                                                                     |
|-----------------------------------------------------|--------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| slip_number                                         | text                     | "device-01-20260708091400" | Unique. Auto-generated from the device ID and a timestamp; a split creates a second, related slip number.               |
| timestamp                                           | timestamp                | 2026-07-08T09:14:00Z       | Dispatch time - the basis for the urgency color-coding.                                                                 |
| supplier_id                                         | number                   | 1                          | Which farm this load's fruit belongs to.                                                                                |
| driver                                              | text                     | "Sello"                    | Required when dispatching.                                                                                              |
| total_crates / total_kg                             | number / decimal         | 18 / 238.5                 |                                                                                                                         |
| status                                              | enum                     | "in_transit"               | One of created (still being picked), in_transit (dispatched), received, processing_complete.                            |
| received_at                                         | timestamp (optional)     | —                          | Set the moment pack house staff confirm receipt.                                                                        |
| weather_temp / weather_humidity / weather_condition | decimal / decimal / text | 24.1 / 55 / "Clear"        | Captured automatically at dispatch time, only if GPS coordinates are set in Settings.                                   |
| split_from_slip_number                              | text (optional)          | —                          | Only set on the "leftover" slip created by a split (chapter 4) - points back at the original slip it was carved out of. |

## ReceivingRecord



| Field                                    | Type   | Example            | Notes / Limitations                                                                                        |
|------------------------------------------|--------|--------------------|------------------------------------------------------------------------------------------------------------|
| lot_id                                   | number | 21                 | Which load this receiving entry belongs to.                                                                |
| expected_crate<br>/<br>actual_crate<br>s | number | 18 / 17            |                                                                                                            |
| discrepancy                              | number | -1                 | Calculated automatically as actual - expected - not entered directly.                                      |
| condition                                | text   | "Good,<br>Sunburn" | Free text, but in practice a comma-joined list of whichever of Good/Damaged/Sunburn/Wet/Other were ticked. |
| received_by                              | text   | "Elsa"             | Required - the app remembers and prefills the last value entered on this device.                           |

## Payment

| Field                        | Type    | Example                    | Notes / Limitations                                                                                                                                               |
|------------------------------|---------|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| worker_id                    | text    | "001"                      |                                                                                                                                                                   |
| period_start /<br>period_end | date    | 2026-07-01 /<br>2026-07-31 |                                                                                                                                                                   |
| total_kg                     | decimal | 318.6                      | Sum of that worker's net crate weights in the period.                                                                                                             |
| rate_applied                 | decimal | 3.00                       | The per-kg rate in effect when calculated.                                                                                                                        |
| amount_due                   | decimal | 955.80                     | Calculated - not directly editable. There is intentionally no "paid" flag or status field on this record (chapter 9); payment status is tracked outside this app. |

## RateSetting

| Field          | Type | Example    | Notes / Limitations       |
|----------------|------|------------|---------------------------|
| effective_date | date | 2026-07-01 |                           |
| rate_type      | enum | "per_kg"   | per_kg or per_crate_tier. |

|                     |             |                                  |                                                                                 |
|---------------------|-------------|----------------------------------|---------------------------------------------------------------------------------|
| default_rate_per_kg | decimal     | 3.00                             | Used when rate_type is per_kg.                                                  |
| tier_rates_json     | text (JSON) | {"1": 2.5, "1.5": 3.5, "2": 4.5} | Only relevant if using per_crate_tier - maps a crate-size tier to its own rate. |

## SystemSetting

| Field                                           | Type               | Example                        | Notes / Limitations                                                     |
|-------------------------------------------------|--------------------|--------------------------------|-------------------------------------------------------------------------|
| farm_name                                       | text               | "Laughing Waters (Bekfontein)" |                                                                         |
| farm_location                                   | text               | "Bekfontein, Mpumalanga"       | Free text description, not used for weather (see gps_lat/gps_lon).      |
| current_harvest_year                            | number             | 2026                           | Drives what "Season" means throughout the app (chapters 7, 9, 10).      |
| green_to_yellow_minutes / yellow_to_red_minutes | number             | 90 / 150                       | The urgency color thresholds referenced throughout chapters 1, 4, 6, 7. |
| gps_lat / gps_lon                               | decimal (optional) | -25.572747 / 31.606722         | Setting both enables automatic weather capture on dispatch.             |

## AdminUser

| Field         | Type | Example  | Notes / Limitations                                       |
|---------------|------|----------|-----------------------------------------------------------|
| username      | text | "admin"  |                                                           |
| password_hash | text | (hashed) | Never shown or exported anywhere - only a hash is stored. |